E-COMMERCE SALES ANALYSIS

PROJECT USING SQL

CHALLENGE TASKS

1. Customer Analysis
2. Order Patterns
3. Top-Selling Products
4. Revenue Breakdown
5. Customer Segmentation
6. Recent Orders
7. Order Value Analysis

🎯 Business Problem Scenario

You work as a **Data Analyst** for an eCommerce platform. The company has been growing, and leadership has asked you to analyze customer behavior, sales trends, and product performance to provide actionable insights. Here are the key business questions that need to be answered:

🔍 Challenge Tasks

1. **Customer Analysis**:
   - Find all customers who have made more than **two orders** in total.
   - List **customer names**, **total amount spent**, and **total number of orders** for each customer who has made purchases. Sort the results by **total amount spent** in descending order.
2. **Product Analysis**:
   - Find all **products** that have been **ordered more than twice**. List the product names, total orders, and total revenue generated for each of these products.
   - Show the **average price** of products for each **product category**.
3. **Top Selling Products**:
   - Identify the **top 3 selling products** by total revenue. Return the product names and the total revenue for each of these products.
4. **Sales Trends**:
   - Find out which **month** has the highest total sales revenue. Include the **month** and **total revenue** in your result.
5. **Customer Segmentation**:
   - Create a **customer segmentation** based on their total spend:
     - **'High Value'**: Total spend > 1000
     - **'Medium Value'**: Total spend between 500 and 1000
     - **'Low Value'**: Total spend < 500
   - Display the **customer name** and their **segmentation category**.
6. **Order Analysis**:
   - Find the **average order value** (i.e., the average price of products per order) for all orders made in **March 2024**.
7. **Cross-Table Analysis**:
   - For each **customer**, list the **products** they ordered along with the **date of order**. You need to use **Joins** to retrieve this data.

SQL File 5* SQL File 6* SQL File 4* SQL File 5*

Limit to 1000 rows

```sql
-- create database pixelfectore_Project_training_1
use pixelfectore_Project_training_1;
select *from customers;
select *from orders;
select *from products;
-- Business Problem Scenario
-- You work as a Data Analyst for an eCommerce platform. The company has been growing, and leade
-- behavior, sales trends, and product performance to provide actionable insights. Here are the
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	name	email
1	Alice Smith	alice@example.com
2	Bob Johnson	bob@example.com
3	Charlie Lee	charlie@example.com
4	David Wilson	david@example.com
5	Eva Green	eva@example.com
6	Frank Thomas	frank@example.com
7	Grace Miller	grace@example.com
8	Hannah Adams	hannah@example.com
9	Will jack	willjack@example.com
10	Stive Smith	StiveSmith@example.com
NULL	NULL	NULL

customers 9

Output

SQL File 5* SQL File 6* SQL File 4* SQL File 5*

Limit to 1000 rows

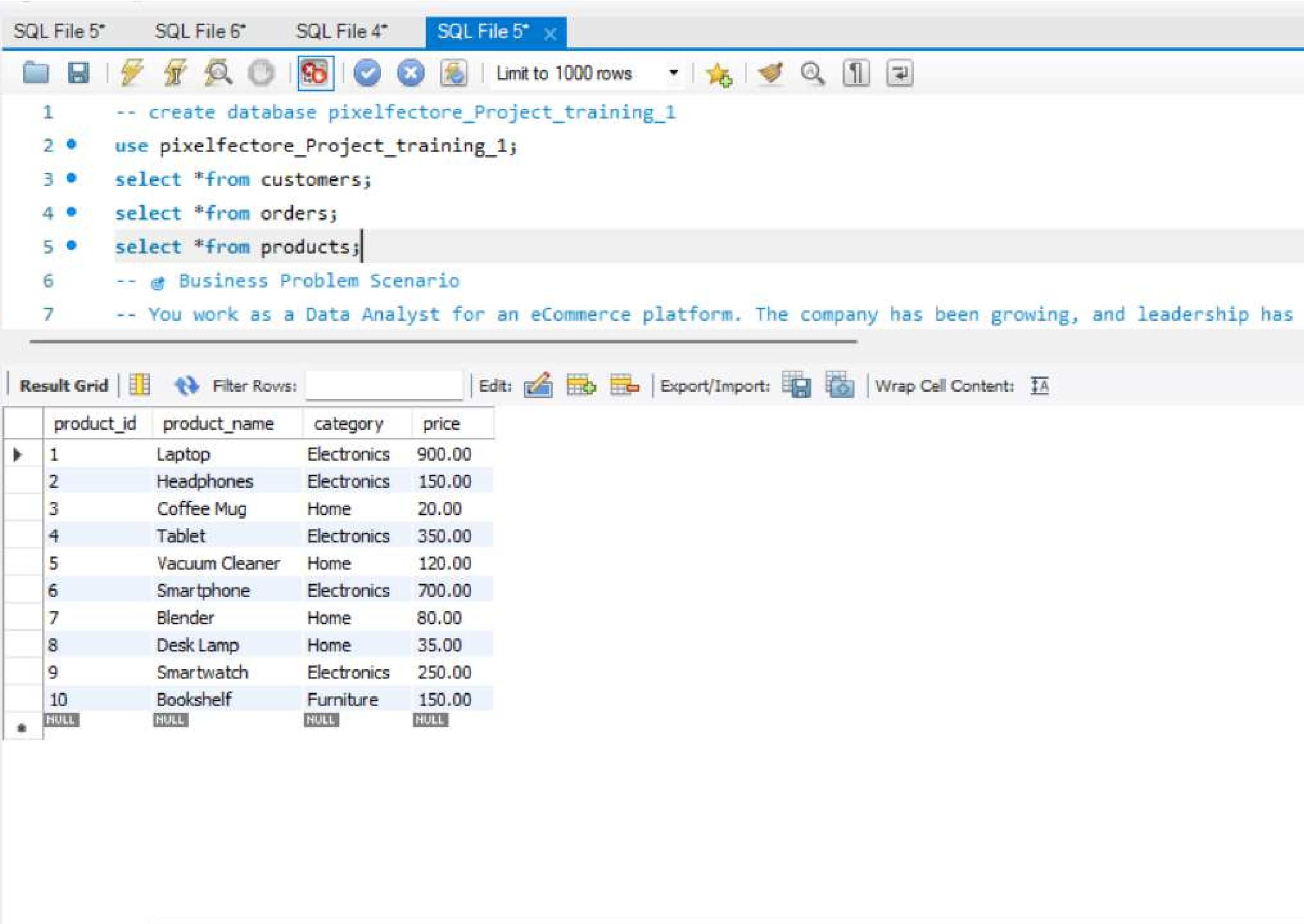
```
-- create database pixelfectore_Project_training_1
use pixelfectore_Project_training_1;
select *from customers;
select *from orders;
select *from products;
-- 🎯 Business Problem Scenario
-- You work as a Data Analyst for an eCommerce platform. The company has been growing, and leadership has
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	customer_id	product_id	order_date
101	1	1	2024-03-01
102	1	2	2024-03-02
103	2	4	2024-03-03
104	3	3	2024-03-04
105	2	5	2024-03-05
106	4	6	2024-03-06
107	5	7	2024-03-07
108	4	2	2024-03-08
109	6	1	2024-03-09
110	7	8	2024-03-10
111	5	9	2024-03-11
112	8	10	2024-03-12
113	6	3	2024-03-13
114	7	4	2024-03-14
115	8	5	2024-03-15
116	1	3	2024-03-16
117	1	5	2024-03-17
NULL	NULL	NULL	NULL

orders 10

Output



SQL File 5* SQL File 6* SQL File 4* SQL File 5*

Limit to 1000 rows

```sql
-- create database pixelfectore_Project_training_1
use pixelfectore_Project_training_1;
select *from customers;
select *from orders;
select *from products;
-- 🎯 Business Problem Scenario
-- You work as a Data Analyst for an eCommerce platform. The company has been growing, and leadership has
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	product_name	category	price
1	Laptop	Electronics	900.00
2	Headphones	Electronics	150.00
3	Coffee Mug	Home	20.00
4	Tablet	Electronics	350.00
5	Vacuum Cleaner	Home	120.00
6	Smartphone	Electronics	700.00
7	Blender	Home	80.00
8	Desk Lamp	Home	35.00
9	Smartwatch	Electronics	250.00
10	Bookshelf	Furniture	150.00
NULL	NULL	NULL	NULL

```sql
-- Business Problem Scenario
-- You work as a Data Analyst for an eCommerce platform. The company has been growing, and leadership has asked you to analyze customer
-- behavior, sales trends, and product performance to provide actionable insights. Here are the key business questions that need to be answere
-- 1.Customer Analysis:-
-- i>Find all customers who have made more than two orders in total.
select
    customers.name
    from customers
    join orders on customers.customer_id = orders.customer_id
    group by customers.customer_id
having count(orders.order_id)>2;
```

name
Alice Smith

```sql
-- ii>List customer names, total amount spent, and total number of orders for each customer who has made purchases. Sort the results
-- by total amount spent in descending order.
select
    customers.name,
    sum(products.price) as Total_AmountSpent,
    count(orders.order_id) as Total_NoOrders
from customers
join orders on customers.customer_id = orders.customer_id
join products on orders.product_id = products.product_id
group by customers.customer_id
order by Total_AmountSpent desc;
```

name	Total_AmountSpent	Total_NoOrders
Alice Smith	1190.00	4
Frank Thomas	920.00	2
David Wilson	850.00	2
Bob Johnson	470.00	2
Grace Miller	385.00	2
Eva Green	330.00	2
Hannah Adams	270.00	2
Charlie Lee	20.00	1

SQL File 5* SQL File 6* SQL File 4* SQL File 5*

Limit to 1000 rows

```sql
-- 2.Product Analysis:-
-- iii>Find all products that have been ordered more than twice. List the product names, total orders, and total revenue
-- generated for each of these products.
select
    products.product_name,
    count(orders.order_id) as Total_Orders,
    sum(price) as Total_Revenue
from products
join orders on products.product_id = orders.product_id
group by products.product_id
having Total_Orders>2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	Total_Orders	Total_Revenue
Coffee Mug	3	60.00
Vacuum Cleaner	3	360.00

Result 14

SQL File 5* SQL File 6* SQL File 4* SQL File 5*

Limit to 1000 rows

```sql
-- iv>Show the average price of products for each product category.
select
    category,
    avg(price) as Avg_Price
    from products
group by category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	Avg_Price
Electronics	470.000000
Home	63.750000
Furniture	150.000000

SQL File 5* SQL File 6* SQL File 4* SQL File 5*

Limit to 1000 rows

```sql
-- 3. Top Selling Products:-
-- v>Identify the top 3 selling products by total revenue. Return the product names and the total revenue for each of these products.
select
   products.product_name,
   sum(products.price) as Total_Revenue
from products
join orders on products.product_id = orders.product_id
group by products.product_id
order by Total_Revenue desc
limit 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

product_name	Total_Revenue
Laptop	1800.00
Smartphone	700.00
Tablet	700.00

```sql
-- 4. Sales Trends:-
-- Find out which month has the highest total sales revenue. Include the month and total revenue in your result.
SELECT
    EXTRACT(MONTH FROM order_date) AS month,
    SUM(products.price) AS total_revenue
FROM orders
JOIN products ON orders.product_id = products.product_id
GROUP BY EXTRACT(MONTH FROM order_date)
ORDER BY total_revenue DESC
LIMIT 1;
```

month	total_revenue
3	4435.00

SQL File 5* SQL File 6* SQL File 4* SQL File 5*

Limit to 1000 rows

```sql
-- 5. Customer Segmentation:-
-- Create a customer segmentation based on their total spend:-
-- 'High Value': Total spend > 1000
-- 'Medium Value': Total spend between 500 and 1000
-- 'Low Value': Total spend < 500
-- Display the customer name and their segmentation category.
SELECT
    customers.name,
    CASE
        WHEN SUM(products.price) > 1000 THEN 'High Value'
        WHEN SUM(products.price) BETWEEN 500 AND 1000 THEN 'Medium Value'
        ELSE 'Low Value'
    END AS customer_segment
FROM customers
JOIN orders ON customers.customer_id = orders.customer_id
JOIN products ON orders.product_id = products.product_id
GROUP BY customers.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	customer_segment
Alice Smith	High Value
Frank Thomas	Medium Value
David Wilson	Medium Value
Charlie Lee	Low Value
Bob Johnson	Low Value
Grace Miller	Low Value
Hannah Adams	Low Value
Eva Green	Low Value

```sql
-- 6. Order Analysis:-
-- Find the average order value (i.e., the average price of products per order) for all orders made in March 2024.
 SELECT
    AVG(products.price) AS avg_order_value
FROM orders
JOIN products ON orders.product_id = products.product_id
WHERE EXTRACT(MONTH FROM order_date) = 3 AND EXTRACT(YEAR FROM order_date) = 2024;
```

avg_order_value
260.882353

SQL File 5* SQL File 6* SQL File 4* SQL File 5*

Limit to 1000 rows

```sql
-- 7. Cross-Table Analysis:-
-- For each customer, list the products they ordered along with the date of order. You need to use Joins to retrieve this data.
select
    customers.name as Customer_Name,
    products.product_name,
    orders.order_date
from customers
join orders on customers.customer_id = orders.customer_id
join products on orders.product_id = products.product_id
order by customers.name,orders.order_date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Customer_Name	product_name	order_date
Alice Smith	Laptop	2024-03-01
Alice Smith	Headphones	2024-03-02
Alice Smith	Coffee Mug	2024-03-16
Alice Smith	Vacuum Cleaner	2024-03-17
Bob Johnson	Tablet	2024-03-03
Bob Johnson	Vacuum Cleaner	2024-03-05
Charlie Lee	Coffee Mug	2024-03-04
David Wilson	Smartphone	2024-03-06
David Wilson	Headphones	2024-03-08
Eva Green	Blender	2024-03-07
Eva Green	Smartwatch	2024-03-11
Frank Thomas	Laptop	2024-03-09
Frank Thomas	Coffee Mug	2024-03-13
Grace Miller	Desk Lamp	2024-03-10
Grace Miller	Tablet	2024-03-14
Hannah Adams	Bookshelf	2024-03-12
Hannah Adams	Vacuum Cleaner	2024-03-15

Result Grid

Form Editor

Field Types

Query Stats

Execution Plan